

Minimum Spanning Trees: Greedy Meets Graphs

Programming and Data Structures — Week 10, Lecture 3

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to define a graph, an edge weight, and a spanning tree precisely, ahead of Week 11's full graph treatment; state the minimum spanning tree problem and explain why both the "spanning" requirement and the "minimum" requirement are essential, neither one alone; implement and trace Kruskal's algorithm completely, including the cycle-detection step it depends on; and explain, at a high level, why Kruskal's greedy edge-by-edge selection is provably optimal.

Outline

Today covers a minimum-cost-road-network analogy motivating the MST problem; graphs, edge weights, and spanning trees, defined precisely ahead of Week 11's full treatment; the MST problem itself, stated precisely; Kruskal's algorithm, which sorts edges and greedily adds them while skipping any that would form a cycle; cycle detection, previewing Week 11's Union-Find structure; a complete trace of Kruskal's algorithm on a concrete example; the MTech-track cut property justifying greedy correctness here; an in-class quiz; and a summary.

The minimum-cost-road-network analogy

Picture a set of towns that must all be connected by roads at minimum total construction cost. Every town must be reachable from every other town, directly or via intermediate towns — but building any redundant road, one that creates an alternative route between towns already connected, only adds cost without adding reachability. This is precisely the minimum spanning tree problem: connect every node in a network at minimum total cost, using no more connections than strictly necessary to maintain full connectivity.

Graphs, edge weights, and spanning trees, defined precisely

A graph, covered formally beginning in Week 11, consists of a set of nodes, or vertices, and a set of edges connecting pairs of them. An edge weight is a cost or distance associated with using that particular edge — a road's construction cost, in this lecture's analogy. A spanning tree is a subset of a graph's edges that connects all n nodes together using exactly $n - 1$ edges, with no cycles at all — exactly enough edges to guarantee connectivity, and not one edge more, directly mirroring Week 7's binary trees, where n nodes require exactly $n - 1$ edges to connect them all with no redundancy.

The MST problem, stated precisely

The minimum spanning tree problem takes as input a connected graph with n nodes and weighted edges, and asks for a spanning tree — exactly $n - 1$ edges connecting every node with no cycles — whose total edge weight is as small as possible among all valid spanning trees. Both requirements are essential: satisfying “spanning” alone is trivial, since almost any $n - 1$ well-chosen edges will connect everything; it is the “minimum” requirement, choosing among potentially many valid spanning trees for the cheapest one, that makes this problem genuinely interesting and non-trivial to solve correctly.

Kruskal's algorithm: the greedy idea

```
def kruskal(n, edges):
    # edges: list of (weight, u, v) tuples
    edges = sorted(edges)                # sort by weight - Week 6's sorting
    parent = list(range(n))              # Union-Find: each node starts as its own group

    def find(x):
        while parent[x] != x:
            x = parent[x]
        return x

    mst = []
    for weight, u, v in edges:
        if find(u) != find(v):           # adding this edge would NOT create a cycle
            mst.append((u, v, weight))
            parent[find(u)] = find(v)    # merge the two groups
    return mst
```

Kruskal's algorithm sorts every edge by weight, cheapest first, directly reusing Week 6's sorting algorithms. It then considers each edge in order, greedily adding it to the growing spanning tree unless doing so would connect two nodes already in the same connected group — which would create a cycle,

violating the spanning-tree requirement. The `parent` array and `find` function implement a simple Union-Find structure, tracking which group each node currently belongs to as edges are added and groups merge.

Why cycles specifically must be avoided

A spanning tree, by definition, has exactly $n - 1$ edges and contains no cycles — the identical fact Week 7 established for binary trees, generalized here to arbitrary graphs. A cycle in a set of selected edges necessarily means at least one edge within that cycle is redundant: removing it cannot disconnect any pair of nodes, since an alternate path around the remainder of the cycle still connects them. Since every edge weight is non-negative, removing any such redundant edge can only reduce or leave unchanged the total weight — it can never increase it, which is exactly why a genuinely minimum spanning tree can never contain a cycle, and why Kruskal's algorithm is correct to reject every edge that would create one.

Cycle detection: why `find` works

Each node begins in its own singleton group, indicated by `parent[x] = x`. Merging two groups redirects one group's representative to point at the other's. The `find` function follows parent pointers repeatedly until reaching a node that points to itself, which is that group's current representative. Two nodes belong to the same group exactly when `find` returns the identical representative for both — and this is precisely the cycle check Kruskal's algorithm needs: an edge between two nodes already in the same group would necessarily create a cycle, since a path between them, via earlier added edges, already exists.

Tracing Kruskal's algorithm completely

Nodes: 0,1,2,3. Edges (weight, u, v): (1,0,1), (2,1,2), (3,0,2), (4,2,3), (5,1,3)
Sorted: (1,0,1), (2,1,2), (3,0,2), (4,2,3), (5,1,3)

(1,0,1): `find(0)=0`, `find(1)=1`, different → ADD. Groups: {0,1},{2},{3}
(2,1,2): `find(1)=0` (via merge), `find(2)=2`, different → ADD. Groups: {0,1,2},{3}
(3,0,2): `find(0)=0`, `find(2)=0` (now same group!) → SKIP (would create a cycle)
(4,2,3): `find(2)=0`, `find(3)=3`, different → ADD. Groups: {0,1,2,3}
(5,1,3): all nodes already in one group → SKIP (only 4 nodes need 3 edges - done)

MST edges: (0,1,w1), (1,2,w2), (2,3,w4). Total weight: 1+2+4=7

Tracing Kruskal's algorithm on a 4-node graph completely: the two cheapest edges, weight 1 and weight 2, are added first, merging nodes 0 and 1, then merging in node 2. The weight-3 edge, between 0 and

2, is skipped, since both nodes are already in the same group — adding it would create a cycle without improving connectivity. The weight-4 edge connects node 3, completing the spanning tree with exactly $n - 1 = 3$ edges. The weight-5 edge is never even needed, since the tree is already complete. Total weight: $1 + 2 + 4 = 7$ — this is the minimum possible total weight connecting all 4 nodes, a claim the next slide justifies.

Tracing find with path-following, step by step

Initial: `parent = [0,1,2,3]` (every node its own representative)

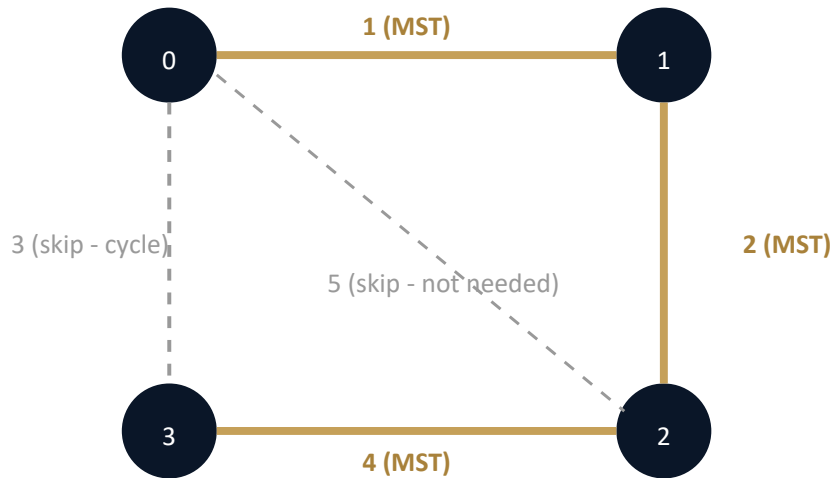
After adding edge (0,1): `parent[find(0)]=find(1) → parent[0]=1 → parent=[1,1,2,3]`

After adding edge (1,2): `parent[find(1)]=find(2) → find(1) walks: parent[1]=1, so find(1)=1`
`parent[1] = find(2) = 2 → parent=[1,2,2,3]`

`find(0)` now: `parent[0]=1, not self → x=1. parent[1]=2, not self → x=2. parent[2]=2, self → r`
`find(0) = 2, matching find(2) = 2 → correctly reports 0 and 2 are in the same group`

Tracing the parent array's evolution and a subsequent `find` call in full detail: after the first two merges, `find(0)` must walk through two intermediate steps — first to node 1, then to node 2 — before reaching a node that points to itself, node 2, the group's current representative. This confirms `find(0)` correctly equals `find(2)`, matching the trace's claim that nodes 0 and 2 are already in the same group by the time the weight-3 edge is considered.

The picture: the graph, and the selected MST edges



Gold edges (weight 1, 2, 4) form the MST, total weight 7. Gray edges are skipped.

Seeing the full graph alongside the selected spanning tree makes the cycle-avoidance concrete: the skipped weight-3 edge would have closed a triangle among nodes 0, 1, and 2 — a redundant connection that adds cost without adding any new reachability.

MTech addition — why greedy works here: the cut property

The cut property states that for any way of partitioning a graph's nodes into two non-empty groups, the minimum-weight edge crossing between the two groups is guaranteed to be part of some minimum spanning tree — an exchange argument (omitted here in full formal detail, but structurally similar to Lecture 1 and Lecture 2's) shows that if it were not, swapping it in for a more expensive crossing edge could only reduce or maintain total weight. Kruskal's algorithm, processing edges from cheapest to most expensive, always adds the current cheapest edge that connects two not-yet-merged groups — which is exactly the edge the cut property guarantees is safe to add. This is a third, genuinely distinct proof, sharing the same underlying exchange-argument technique as Lecture 1 and Lecture 2, but tailored to this problem's own specific structure.

Cost of Kruskal's algorithm

Kruskal's algorithm costs $O(m \log m)$ to sort the m edges, directly reusing Week 6's sorting algorithms. Each `find` or `union` operation, once path compression is applied (this lecture's exercise), costs close to $O(1)$ in practice — technically $O(\alpha(n))$, where α is the inverse Ackermann function, a function that grows so slowly it is effectively constant for any input size that could ever occur in practice. The total cost is therefore $O(m \log m)$, dominated entirely by the initial sort — exactly the same cost-dominance pattern Lecture 1 established for activity selection, where sorting also dominated a cheap linear scan.

Exercise

As an exercise, trace Kruskal's algorithm by hand on a 5-node graph of your own construction, with at least 6 candidate edges, confirming that exactly $n - 1 = 4$ edges are selected in the final spanning tree. Separately, implement `find` with path compression — during the walk following parent pointers to the representative, redirect every visited node to point directly at that final representative — and explain, in a sentence, why this speeds up future `find` calls on the same nodes.

Real-world use of minimum spanning trees

Minimum spanning trees are genuinely used in practice, not merely as a textbook exercise. Network design uses MST algorithms to lay minimum-cost cable or fiber connecting a set of physical locations with no redundant links. Circuit board design uses closely related algorithms to minimize total wire length connecting a set of components. Approximation algorithms for considerably harder problems — the traveling salesman problem, for instance, which has no known efficient exact solution — often use a minimum spanning tree as a starting building block, since it can be computed quickly and provides a provably good lower bound on the harder problem's true optimal cost.

Quiz — will this edge be added?

Given that nodes 2 and 5 are already in the same connected group at some point during Kruskal's algorithm, and the next edge under consideration connects node 2 directly to node 5, determine before checking the answer whether this edge is added to the spanning tree.

A second worked trace, with a tie-breaking scenario

A second trace, on a 3-node graph with two edges tying at weight 2, confirms the algorithm handles ties correctly regardless of the specific tie-breaking rule used when sorting: both weight-2 edges are added first, since neither creates a cycle, completing the spanning tree with exactly $n - 1 = 2$ edges. The weight-3 edge is never even needed, and a practical implementation can stop as soon as $n - 1$ edges have been selected, rather than processing every remaining edge unnecessarily.

Quiz — answer

This edge is not added. Since nodes 2 and 5 are already in the same group, `find(2)` and `find(5)` return the identical representative, meaning a path between them already exists using earlier, cheaper edges already processed. Adding a direct edge between them now would create a cycle, connecting two already-connected nodes redundantly and adding cost with no benefit to overall connectivity. This is exactly what the algorithm's `if find(u) != find(v)` check correctly detects and skips.

A closing note: Prim's algorithm, a different route to the same answer

Kruskal's algorithm processes edges globally, cheapest first, using Union-Find specifically to detect and avoid cycles. Prim's algorithm, a close relative covered in Week 12 alongside Dijkstra's shortest-path algorithm, takes a different route to the same guaranteed-minimum answer: it grows a single connected tree outward from an arbitrary starting node, always adding whichever edge is cheapest among those currently leaving the growing tree — a min-heap, directly reusing Week 8's structure, replaces Union-Find here. Both algorithms are greedy, both are provably optimal via their own respective exchange arguments, and both cost roughly $O(m \log n)$ — two genuinely different mechanisms converging on the identical correct answer, worth knowing both exist even though this lecture covers only Kruskal's in full.

Summary

A minimum spanning tree connects all n nodes of a graph using exactly $n - 1$ edges, at the minimum possible total weight, with no cycles. Kruskal's algorithm achieves this by sorting all edges cheapest-first, directly reusing Week 6's sorting algorithms, and greedily adding each edge unless it would form a cycle, tracked via a simple Union-Find structure. The cut property justifies this greedy strategy formally — a third genuinely distinct exchange argument, following Lecture 1's and Lecture 2's, completing this unit's trio of proven-correct greedy algorithms. This is this course's first genuine graph algorithm, directly previewing Week 11's full formal treatment of graphs. Next lecture consolidates this entire unit's proof technique and builds the judgment needed to recognize when greedy reasoning is, and is not, the appropriate tool for a new optimization problem.